

BlueRaffleBlockhash

Security Audit Report

Comprehensive Exploit Testing & Vulnerability Assessment
For Mainnet Deployment

Date: December 4, 2025
Version: 2.0.0
Network: BSC (Binance Smart Chain)
Solidity Version: 0.8.20

RESULT: 80/80 TESTS PASSING

Contract is ready for mainnet deployment

1. Executive Summary

This security audit was performed on the BlueRaffleBlockhash smart contract to identify vulnerabilities and ensure the contract is secure for mainnet deployment. The audit included:

- 80 automated exploit tests
- Reentrancy attack simulations
- Access control verification
- Fund handling validation
- Randomness fairness analysis
- Edge case testing
- Gas limit / DoS testing

Key Findings

During testing, 3 critical vulnerabilities were discovered and fixed:

- Single Player Fund Lock (HIGH) - FIXED
- Incomplete Refund on Cancel (HIGH) - FIXED
- Refund Math Error (MEDIUM) - FIXED

Final Status

After implementing fixes, all 80 security tests pass. The contract is now significantly more secure and ready for mainnet deployment with the understanding of documented considerations.

2. Vulnerabilities Found & Fixed

2.1 Single Player Fund Lock (HIGH SEVERITY)

Location:

`BlueRaffleBlockhash.sol:184-191`

Issue:

If only one player joined a round, the round stayed in "Waiting" status. The single player could not call `requestDraw()` because it required `status == Active`. Their funds were permanently trapped unless the owner manually called `cancelRound()`.

Impact:

User funds could be locked indefinitely if owner was unavailable or unresponsive.

Fix Applied:

Added new `withdrawFromWaiting()` function allowing players to exit a Waiting round and receive their proportional refund (94% of deposit from prize pool).

```
function withdrawFromWaiting() external nonReentrant
```

2.2 Incomplete Refund on Cancel (HIGH SEVERITY)

Location:

`BlueRaffleBlockhash.sol:346-355`

Issue:

`cancelRound()` attempted to refund 100% of original deposit (tickets / multiplier), but the contract only held ~94% after distribution (2% dev, 2% burn, 1% treasury, 1% seed were already transferred out).

Impact:

Later players in the refund loop would receive nothing because contract balance was insufficient. First players got full refund, later players got zero.

Fix Applied:

Changed refund logic to use proportional share of prize pool:

```
refundAmount = (tickets * prizePoolSnapshot) / totalTicketsSnapshot
```

All players now receive fair 94% refunds proportional to their ticket holdings.

2.3 Refund Math in Single Player Case (MEDIUM SEVERITY)

Location:

`BlueRaffleBlockhash.sol:529-551`

Issue:

`_refundSinglePlayer()` used `tickets / ticketMultiplier` which could produce incorrect amounts with bonus multipliers and didn't account for the distributed 6%.

Fix Applied:

Refund now returns entire prize pool to single player:

```
uint256 refundAmount = round.prizePool;
```

3. Code Improvements Made

3.1 New Function: withdrawFromWaiting()

Added a new public function allowing players to exit rounds that haven't activated yet:

```
function withdrawFromWaiting() external nonReentrant {
    Round storage round = rounds[currentRoundId];
    if (round.status != RoundStatus.Waiting) revert RoundNotWaiting();
    uint256 tickets = userTickets[currentRoundId][msg.sender];
    if (tickets == 0) revert NoTicketsToWithdraw();
    uint256 refundAmount = (tickets * round.prizePool) / round.totalTickets;
    userTickets[currentRoundId][msg.sender] = 0;
    round.totalTickets -= tickets;
    round.prizePool -= refundAmount;
    round.uniqueWallets--;
    if (refundAmount > 0) {
        blueToken.transfer(msg.sender, refundAmount);
        emit RoundRefunded(currentRoundId, msg.sender, refundAmount);
    }
    if (round.uniqueWallets == 0) {
        round.status = RoundStatus.Cancelled;
        _startNewRound();
    }
}
```

3.2 New Error Types Added

```
error NoTicketsToWithdraw();
error RoundNotWaitingOrActive();
```

3.3 Updated cancelRound() Logic

Changed from attempting 100% refund to proportional prize pool refund:

```
uint256 refundAmount = (tickets * prizePoolSnapshot) / totalTicketsSnapshot;
```

3.4 Updated _refundSinglePlayer() Logic

Changed from ticket-based calculation to prize pool refund:

```
uint256 refundAmount = round.prizePool; // Refund entire prize pool
```

4. Complete Test List (80 Tests)

All tests executed with Hardhat test framework on local network.

4.1 Access Control Exploits (10 tests)

- ' EXPLOIT: Non-owner cannot pause contract
- ' EXPLOIT: Non-owner cannot unpause contract
- ' EXPLOIT: Non-owner cannot set bonus multiplier
- ' EXPLOIT: Non-owner cannot set entry limits
- ' EXPLOIT: Non-owner cannot set distribution
- ' EXPLOIT: Non-owner cannot set round duration
- ' EXPLOIT: Non-owner cannot set treasury wallet
- ' EXPLOIT: Non-owner cannot set developer wallet
- ' EXPLOIT: Non-owner cannot cancel round
- ' SECURITY: Owner can perform all admin functions

4.2 Reentrancy Attack Tests (4 tests)

- ' EXPLOIT: Reentrancy during buyTickets blocked by nonReentrant
- ' EXPLOIT: Reentrancy during requestDraw blocked
- ' EXPLOIT: Reentrancy during executeDraw blocked
- ' EXPLOIT: Cannot call buyTickets recursively

4.3 Draw Manipulation Exploits (8 tests)

- ' EXPLOIT: Cannot request draw before timer ends
- ' EXPLOIT: Cannot execute draw without requesting first
- ' EXPLOIT: Cannot execute draw too early (< 2 blocks)
- ' EXPLOIT: Cannot execute draw too late (> 250 blocks)
- ' EXPLOIT: Cannot request draw twice
- ' EXPLOIT: Cannot buy tickets during Drawing phase
- ' EXPLOIT: Cannot buy tickets during Complete phase
- ' SECURITY: Draw executes correctly within valid block window

- ' SECURITY: Draw works at boundary (249 blocks after target)

4.4 Fund Handling Exploits (7 tests)

- ' EXPLOIT: Cannot buy tickets without approval
- ' EXPLOIT: Cannot buy tickets without balance
- ' SECURITY: Distribution percentages are accurate
- ' SECURITY: Winner receives full prize pool
- ' SECURITY: Seed pool carries to next round
- ' EXPLOIT: Cannot steal funds via distribution manipulation
- ' EXPLOIT: Contract balance is properly managed

4.5 Entry Limit Exploits (7 tests)

- ' EXPLOIT: Cannot buy below minimum tickets
- ' EXPLOIT: Cannot buy above maximum tickets in single tx
- ' EXPLOIT: Cannot exceed maximum tickets across multiple purchases
- ' SECURITY: Can buy exactly at minimum
- ' SECURITY: Can buy exactly at maximum
- ' EXPLOIT: Multiplier affects ticket limits correctly
- ' EXPLOIT: Cannot bypass limits via zero amount

4.6 Edge Cases (10 tests)

- ' EDGE: Single player can withdraw from Waiting round
- ' EDGE: Single player refund via requestDraw (after activation)
- ' EDGE: Refund calculation is proportional from prize pool
- ' EDGE: Round cancelled refunds proportionally from prize pool
- ' EDGE: Cannot cancel completed round
- ' EDGE: Paused contract blocks new entries
- ' EDGE: Pause does not affect ongoing draw
- ' EDGE: Multiple sequential rounds work correctly
- ' EDGE: Round activation on exactly 2 players
- ' EDGE: Timer expires exactly at endTime

4.7 Randomness Fairness Tests (4 tests)

- ' SECURITY: Winner selection uses blockhash correctly
- ' SECURITY: Different blocks produce different winners (probabilistically)
- ' SECURITY: Weighted selection favors larger ticket holders
- ' SECURITY: Seed includes multiple entropy sources

4.8 Configuration Validation (6 tests)

- ' EXPLOIT: Cannot set zero addresses
- ' EXPLOIT: Cannot set invalid multiplier
- ' SECURITY: Multiplier boundaries work
- ' EXPLOIT: Cannot set invalid round duration
- ' SECURITY: Round duration boundaries work
- ' EXPLOIT: Cannot set invalid entry limits

4.9 Constructor Validation (4 tests)

- ' EXPLOIT: Cannot deploy with zero token address
- ' EXPLOIT: Cannot deploy with zero treasury address
- ' EXPLOIT: Cannot deploy with zero developer address
- ' SECURITY: Correct initial state after deployment

4.10 Gas Limit / DoS Tests (2 tests)

- ' SECURITY: Many participants don't cause out-of-gas in winner selection
- ' SECURITY: Many participants don't cause out-of-gas in cancel

4.11 View Function Tests (5 tests)

- ' SECURITY: canRequestDraw returns correct status
- ' SECURITY: canExecuteDraw returns correct status
- ' SECURITY: getCurrentRoundInfo returns accurate data
- ' SECURITY: getUserTickets returns correct amount
- ' SECURITY: getRoundParticipants returns correct list

4.12 WithdrawFromWaiting Tests (5 tests)

- ' EXPLOIT: Cannot withdraw from Active round
- ' EXPLOIT: Cannot withdraw without tickets
- ' SECURITY: Withdraw reduces round state correctly
- ' SECURITY: Multiple players can withdraw sequentially
- ' SECURITY: Reentrancy blocked on withdrawFromWaiting

4.13 Event Emission Tests (8 tests)

- ' SECURITY: TicketsPurchased event emitted correctly
- ' SECURITY: RoundActivated event emitted on 2nd player
- ' SECURITY: DrawRequested event emitted correctly
- ' SECURITY: WinnerSelected event emitted correctly
- ' SECURITY: RoundRefunded event emitted on cancel
- ' SECURITY: RoundRefunded event emitted on withdrawFromWaiting
- ' SECURITY: BonusRoundActivated event emitted

5. Remaining Considerations for Mainnet

5.1 Blockhash Randomness

The contract uses blockhash for randomness generation. This is a common pattern but has limitations:

- Blockhash is deterministic once the block is mined
- Validators/miners could theoretically influence outcomes
- Suitable for casual gaming with moderate stakes

Recommendation:

For high-stakes deployment, consider integrating Chainlink VRF for cryptographically secure randomness.

5.2 Pause Mechanism Scope

The whenNotPaused modifier only applies to buyTickets(). The requestDraw() and executeDraw() functions can still be called when paused.

Rationale:

This is intentional - it allows completing ongoing draws during emergency pause while preventing new entries. Document this behavior for operators.

5.3 6% Distribution Model

6% of each deposit is distributed immediately:

- 2% to Developer Wallet
- 2% to Burn Address
- 1% to Treasury Wallet
- 1% to Seed Pool

Players receive 94% refund on cancellation/withdrawal. This is working as designed but should be clearly communicated to users.

5.4 Owner Centralization

All administrative functions are controlled by a single owner address:

- Pause/Unpause
- Set multipliers and limits
- Change distribution percentages
- Cancel rounds
- Update wallet addresses

Recommendation:

For mainnet, consider using a multi-signature wallet (Gnosis Safe) for owner address to prevent single point of failure.

6. Conclusion

The BlueRaffleBlockhash smart contract has undergone comprehensive security testing with 80 exploit tests covering:

- Access control mechanisms
- Reentrancy attack vectors
- Draw manipulation attempts
- Fund handling and distribution
- Entry limit enforcement
- Edge cases and boundary conditions
- Randomness fairness
- Configuration validation
- Gas limits and DoS resistance

Critical Issues: RESOLVED

Three critical vulnerabilities were discovered and fixed during the audit:

1. **Single Player Fund Lock - FIXED**
2. **Incomplete Refund on Cancel - FIXED**
3. **Refund Math Error - FIXED**

Final Verdict

CONTRACT IS READY FOR MAINNET DEPLOYMENT

With the fixes applied and all 80 tests passing, the contract demonstrates robust security measures including:

- ReentrancyGuard on all state-changing functions
- Pausable emergency controls
- Access control via Ownable
- Comprehensive input validation
- Secure fund distribution
- Fair weighted random selection
- Proper state management

This audit report was generated as part of security testing for the BlueRaffleBlockhash smart contract.
Always conduct additional manual review and consider professional third-party audits before mainnet deployment.